MoovUp

API RESTful para converter pontos de engajamento em tickets de transporte público

Domain Driven Design Using Java, Challenge 2026, Sprint 4

Equipe: MoovUp
Turma: 1TDSPJ

Integrante	**RM**
Juan Souza Marques	573469
Lucas Leite Carlos	571985
Matheus Matsushita de Souza	570017
Pedro Amaro Pires	570636

API publicada: https://challenge-2026-java-sprint-4.onrender.com
Documentação interativa (Swagger):
https://challenge-2026-java-sprint-4.onrender.com/swagger-ui.html
Repositório: https://github.com/challenge-2026-MoovUp/challenge-2026-java-sprint-4

Contexto

A SoulUp é uma plataforma de engajamento digital que recompensa seus usuários com pontos por ações realizadas dentro do ambiente online. O desafio é tornar esses pontos úteis fora da plataforma, de forma acessível e com impacto social.
O transporte público é uma das principais formas de reduzir a emissão de carbono nos centros urbanos. Em São Paulo, onde milhões de pessoas usam o Metrô e a CPTM todos os dias, transformar pontos em passagens é um benefício concreto e recorrente, e permite medir quanto CO2 deixou de ser emitido.

Objetivo e escopo

O MoovUp permite que o usuário acumule pontos com a sua participação na rede social (postagens e viagens registradas), troque esses pontos por tickets de metrô ou trem e acompanhe o impacto ambiental das suas viagens.
A solução trabalha com estoque próprio de tickets: a plataforma mantém tickets digitais disponíveis e, no resgate, entrega ao usuário um ticket com QR Code ou recarga. A integração direta com o bilhete único (TOP/SPTrans) fica como evolução futura, porque depende de parceria com as operadoras.
Nesta sprint, a aplicação deixou de ser um programa de console e passou a ser uma **API RESTful com Spring Boot**, publicada na nuvem e consumida pelo front-end em React. O escopo da API cobre:

- cadastro, login, alteração, consulta e exclusão de usuários;
- postagens, comentários e curtidas, com geração de pontos;

- seguir e deixar de seguir outros usuários;
- gestão do estoque de tickets e resgate de tickets com pontos;
- registro de viagens por linha e estação, com geração de pontos e de impacto ambiental;
- ranking de pontos e resumo do impacto ambiental de cada usuário.

Limites da solução: é um projeto acadêmico. A autenticação é simples (e-mail e senha, sem token), a senha é guardada sem criptografia e não há integração com sistemas reais de bilhetagem.

Funcionalidades

Usuários

Cadastro com nome, e-mail e senha; login por e-mail e senha; consulta por id ou por nome; alteração de nome e e-mail; consulta do saldo de pontos; ranking dos usuários ativos por saldo. A exclusão é lógica: o usuário passa a ter status inativo, deixa de aparecer nas listagens e não consegue fazer login, mas o histórico de tickets e viagens continua no banco.

Postagens, comentários e curtidas

O usuário publica postagens de texto, imagem ou vídeo e recebe pontos de acordo com o tipo. Outros usuários podem comentar e curtir; cada usuário curte uma postagem uma única vez. Se a postagem for excluída, os pontos que ela gerou são retirados do autor.

Seguir usuários

O usuário pode seguir outra pessoa pelo id ou pelo nome, listar quem ele segue e deixar de seguir. Não é possível seguir a si mesmo nem seguir a mesma pessoa duas vezes.

Tickets e resgates

O estoque é abastecido com tickets de metrô ou trem. No resgate, a API pega um ticket disponível da modalidade pedida, confere o saldo do usuário, desconta os pontos, define a forma de recebimento (QR Code ou recarga), marca o ticket como reservado e define a validade de dois dias. Na catraca, o ticket é marcado como utilizado.

Viagens e impacto ambiental

Depois de usar o ticket, o usuário registra a viagem informando a linha e a estação de embarque. A API confere se a estação pertence à linha, dá 35 pontos ao usuário e grava o impacto ambiental da viagem. As 15 linhas do Metrô e da CPTM e as suas estações estão cadastradas na aplicação e podem ser consultadas pela API.

Regras de negócio

Regra	Valor
Pontos por postagem	Texto 50, imagem 100, vídeo 150
Pontos por viagem registrada	35 (uma viagem por ticket)
Custo do ticket	Tarifa dividida por 0,009, arredondada para baixo: R$ 5,00 equivale a 555 pontos (metrô e trem)
Validade do ticket	2 dias a partir do resgate; vencido, passa para EXPIRADO
Ciclo do ticket	DISPONIVEL, depois RESERVADO (resgate), depois UTILIZADO (catraca); ou EXPIRADO
Impacto por viagem	Metrô: 10 km; trem: 20 km; 200 g de CO2 evitado por km
Abastecimento do estoque	De 1 a 50 tickets por pedido
Saldo de pontos	Nunca fica negativo

Todas as validações e regras de negócio ficam na camada de serviço (BO). Os models guardam o estado e as ações que o alteram, e os DAOs só acessam o banco.

Arquitetura da aplicação

O projeto segue o padrão em camadas MVC com DAO, no pacote br.com.fiap.MoovUp:

Pacote	Responsabilidade
models	Classes de domínio: Usuario, Postagem, Comentario, Curtida, Seguidor, Ticket (abstrata), TicketDigital, Viagem e ImpactoAmbiental. Os relacionamentos são feitos por objeto (por exemplo, Postagem tem um Usuario).
interfaces	Resgatavel, implementada pelos tickets (resgatar e calcularCustoEmPontos).
enums	TipoPostagem, Modalidade, StatusTicket, StatusUsuario, FormaRecebimento, Linha e Estacao, com valores fixos como pontos, tarifa, km e CO2 por km.
dao	ConnectionFactory e um DAO por tabela, com JDBC puro e PreparedStatement.
service	Camada de regras de negócio (BO): validações, cálculo de pontos, resgate,

	viagens e impacto.
controller	Camada Resource: os endpoints REST, com os verbos e os status HTTP.
dto	Records de entrada (Request) e saída (Response) da API. A senha nunca aparece nas respostas.
mapper	Conversão entre models e DTOs (toEntity e toDTO).
exception	Exceções da aplicação e a classe TratadorDeErros.
config	CorsFilter, que libera o acesso do front-end publicado em outro domínio.

Tratamento de exceções

Os services lançam uma exceção específica para cada tipo de erro. Cada endpoint captura a exceção e a classe TratadorDeErros escolhe o status HTTP, devolvendo sempre o mesmo formato de resposta: status, mensagem e data e hora.

Exceção	**Status**	**Quando ocorre**
IllegalArgumentException	400 Bad Request	Dados inválidos ou faltando (campo obrigatório, tamanho, valor fora da lista)
CredenciaisInvalidasException	401 Unauthorized	E-mail ou senha incorretos no login
RecursoNaoEncontradoException	404 Not Found	Usuário, postagem, comentário, ticket ou linha não encontrados
RegraNegocioException	422 Unprocessable Entity	Regra de negócio violada (saldo insuficiente, estoque vazio, e-mail já cadastrado, curtida repetida)
Qualquer outra	500 Internal Server Error	Falha inesperada, por exemplo na conexão com o banco

Exemplo de resposta de erro:

```
{ "status": 404, "mensagem": "Usuário não encontrado", "timestamp": "2026-11-04T14:30:00" }
```

Endpoints

Todos os endpoints recebem e devolvem JSON. Os identificadores são texto (UUID). Além dos erros listados em cada linha, qualquer endpoint pode devolver 500 em caso de falha inesperada.

Usuários

Verbo	URI	Corpo	Sucesso	Erros
POST	/usuarios	nome, email, senha	201	400, 422
POST	/usuarios/login	email, senha	200	400, 401
GET	/usuarios		200	
GET	/usuarios/ranking		200	
GET	/usuarios/nome/{nome}		200	404
GET	/usuarios/{id}		200	404
PUT	/usuarios/{id}	nome, email	200	400, 404, 422
DELETE	/usuarios/{id}		204	404
GET	/usuarios/{id}/pontos		200	404
GET	/usuarios/{id}/impacto		200	404
GET	/usuarios/{id}/impacto/historico		200	404

Seguir usuários

Verbo	URI	Corpo	Sucesso	Erros
GET	/usuarios/{id}/seguindo		200	404
POST	/usuarios/{id}/seguindo	idSeguido ou nomeSeguido	201	400, 404, 422
DELETE	/usuarios/{id}/seguindo/{idSeguido}		204	404

Postagens e curtidas

Verbo	URI	Corpo	Sucesso	Erros
POST	/postagens	idUsuario, titulo, conteudo, imagem, video, tipoPostagem	201	400, 404
GET	/postagens		200	
GET	/postagens/{id}		200	404
PUT	/postagens/{id}	titulo, conteudo	200	400, 404
DELETE	/postagens/{id}		204	404
POST	/postagens/{id}/curtid	idUsuario	201	404, 422

	as			

Comentários

Verbo	URI	Corpo	Sucesso	Erros
POST	/postagens/{id}/comentarios	idUsuario, conteudo	201	400, 404
GET	/postagens/{id}/comentarios		200	404
DELETE	/comentarios/{id}		204	404

Tickets (estoque)

Verbo	URI	Corpo	Sucesso	Erros
POST	/tickets	modalidade, quantidade	201	400
GET	/tickets?status=&modalidade=		200	400
GET	/tickets/estoque		200	
GET	/tickets/{id}		200	404
PATCH	/tickets/{id}/uso		200	404, 422
DELETE	/tickets/{id}		204	404, 422

Resgates

Verbo	URI	Corpo	Sucesso	Erros
POST	/usuarios/{id}/resgates	modalidade, formaRecebimento	201	400, 404, 422
GET	/usuarios/{id}/resgates		200	404

Viagens, linhas e estações

Verbo	URI	Corpo	Sucesso	Erros
POST	/usuarios/{id}/viagens	idTicket, linha, estacaoEmbarque	201	400, 404, 422
GET	/usuarios/{id}/viagens		200	404
GET	/linhas?modalidade=		200	400
GET	/linhas/{codigo}/estacoes		200	404

São 35 endpoints no total. Exemplos de corpo:

```
POST /usuarios
{ "nome": "Lucas Leite", "email": "lucas.leite@moovup.com", "senha":
"Moovup123" }
```

```
POST /usuarios/{id}/resgates
{ "modalidade": "METRO", "formaRecebimento": "QR_CODE" }

POST /usuarios/{id}/viagens
{ "idTicket": "TCK-001", "linha": "AZUL", "estacaoEmbarque": "JABAQUARA" }
```

Telas do front-end

As telas abaixo são do front-end publicado pelo grupo (React), que consome a API publicada no Render. Os dados que aparecem nelas, como o saldo de 180 pontos e o ranking, vêm do banco Oracle por meio dos endpoints indicados em cada legenda.

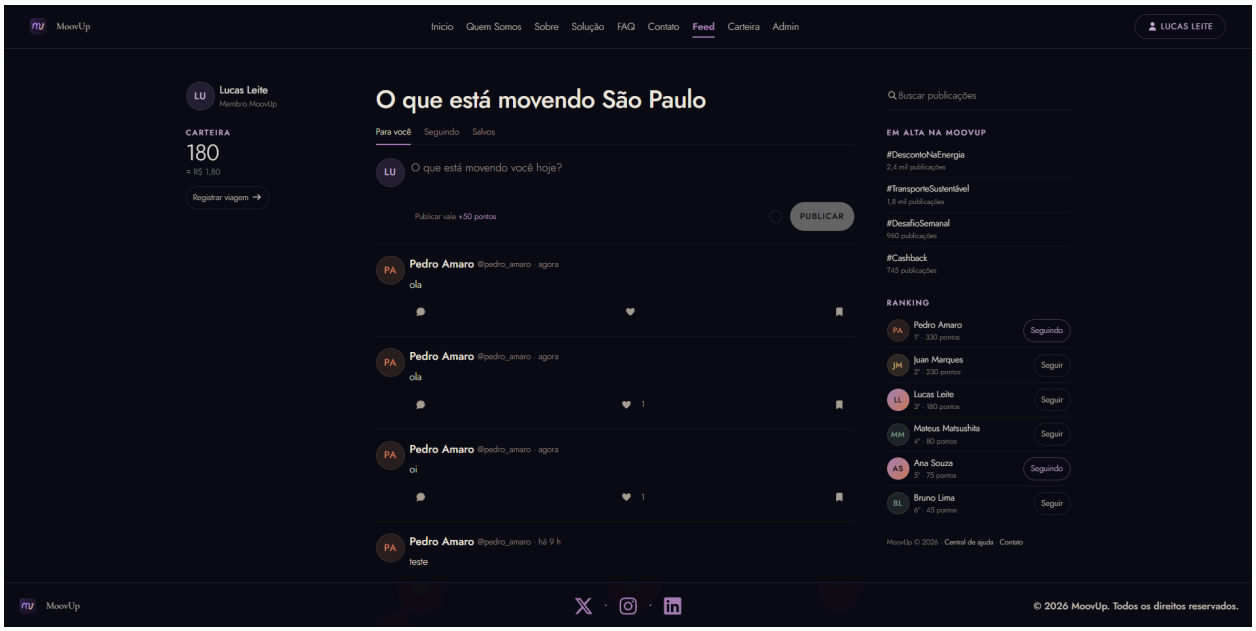


Figura 1: Feed. *Lista as postagens (GET /postagens), com curtidas e comentários, e permite publicar uma nova postagem (POST /postagens). Ao lado ficam o saldo de pontos do usuário e o ranking (GET /usuarios/ranking), com o botão para seguir outros usuários (POST /usuarios/{id}/seguindo).*

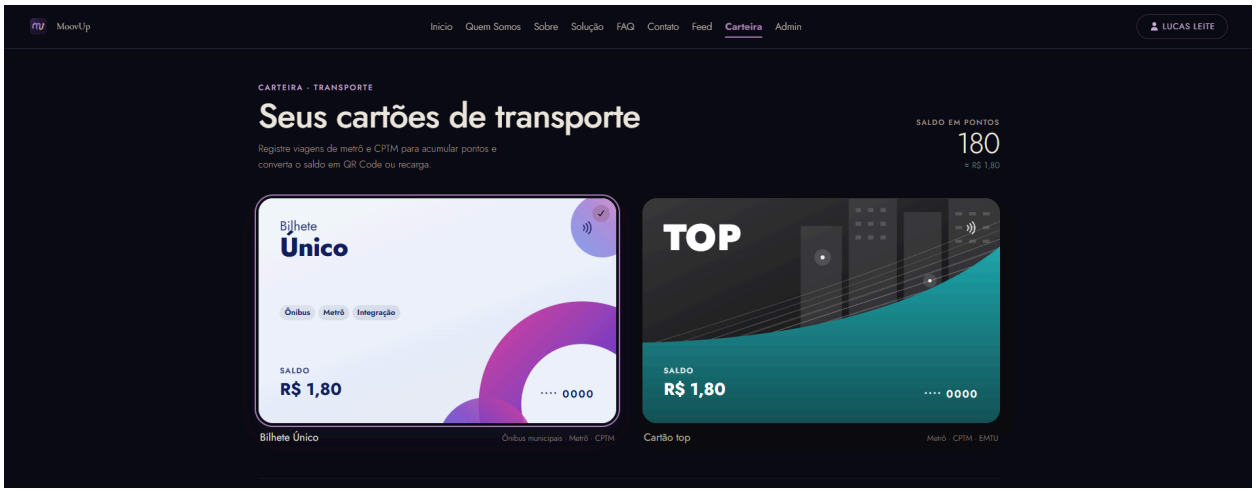


Figura 2: Carteira. *Mostra o saldo de pontos do usuário logado (GET /usuarios/{id}/pontos) e os cartões de transporte do usuário.*

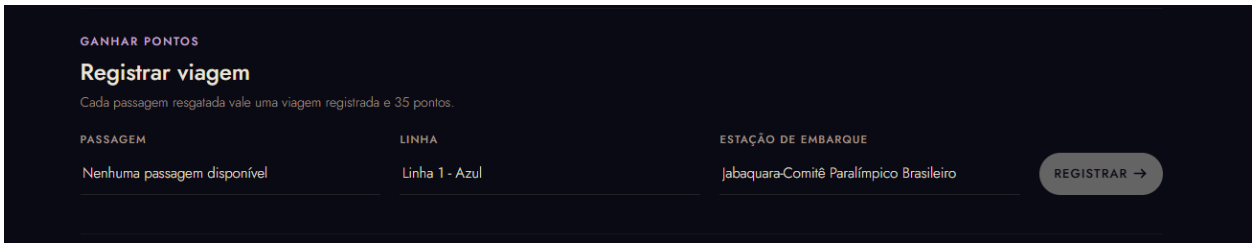


Figura 3: Registrar viagem. *O usuário escolhe uma passagem já usada na catraca, a linha e a estação de embarque. As linhas e estações vêm de GET /linhas e GET /linhas/{codigo}/estacoes. Ao registrar (POST /usuarios/{id}/viagens), o usuário ganha 35 pontos e o impacto ambiental da viagem é gravado.*

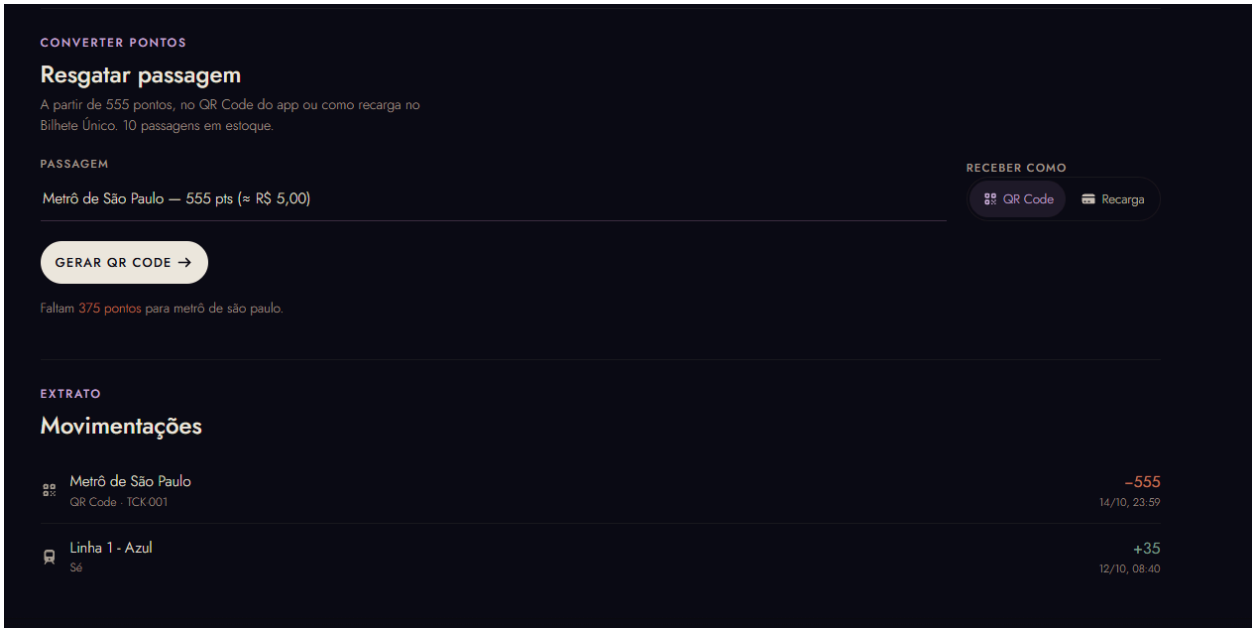


Figura 4: Resgatar passagem e extrato. *O usuário escolhe a passagem e a forma de recebimento (QR Code ou recarga) e faz o resgate (POST /usuarios/{id}/resgates). A tela mostra quantos pontos faltam e quantas passagens há no estoque. O extrato junta os resgates (GET /usuarios/{id}/resgates, 555 pontos a menos) e as viagens (GET /usuarios/{id}/viagens, 35 pontos a mais).*

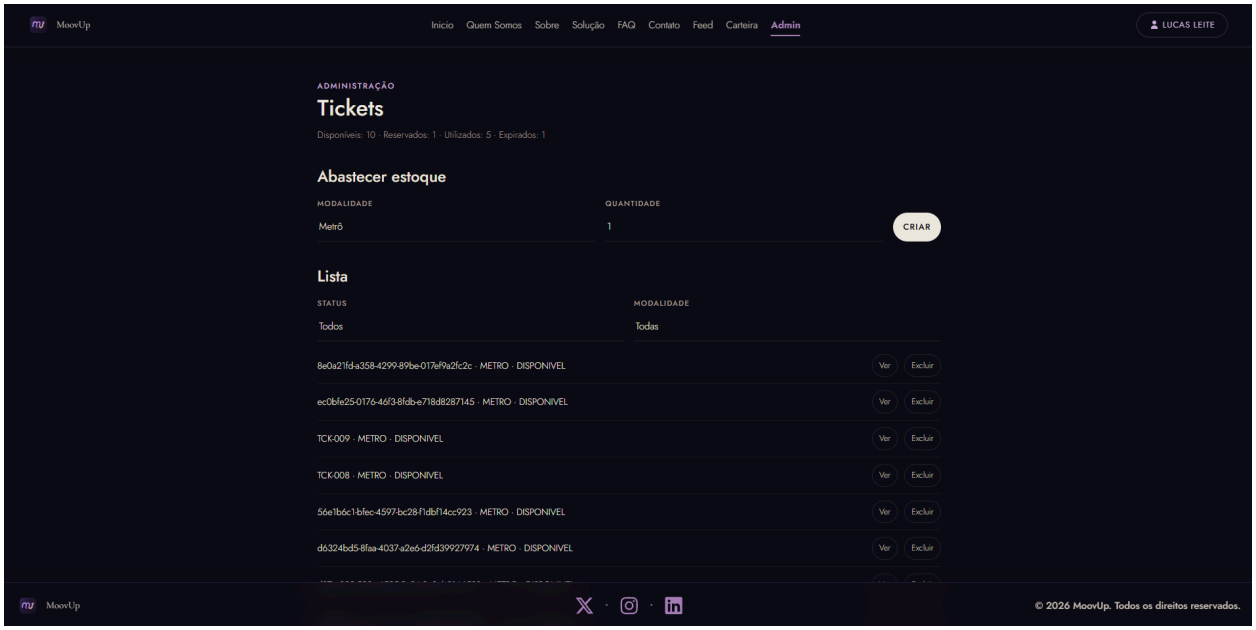


Figura 5: Administração de tickets. *Mostra o resumo do estoque por status (GET /tickets/estoque), abastece o estoque por modalidade e quantidade (POST /tickets) e lista os tickets com filtro de status e modalidade (GET /tickets), com as opções de ver (GET /tickets/{id}) e excluir (DELETE /tickets/{id}).*

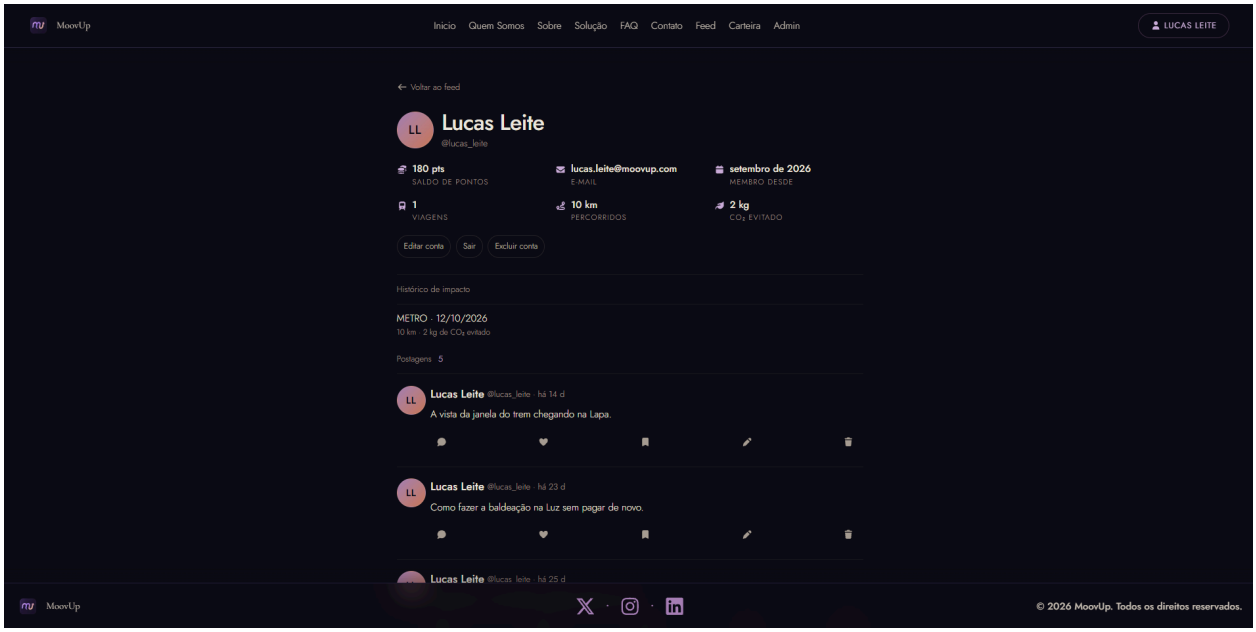


Figura 6: Perfil. *Mostra os dados do usuário (GET /usuarios/{id}), o saldo de pontos, o resumo do impacto ambiental com viagens, quilômetros e CO2 evitado (GET /usuarios/{id}/impacto) e o histórico de impacto (GET /usuarios/{id}/impacto/historico). Lista as postagens do usuário, que podem ser editadas (PUT /postagens/{id}) ou excluídas (DELETE /postagens/{id}), e tem os botões para editar a conta (PUT /usuarios/{id}) e excluí-la (DELETE /usuarios/{id}).*

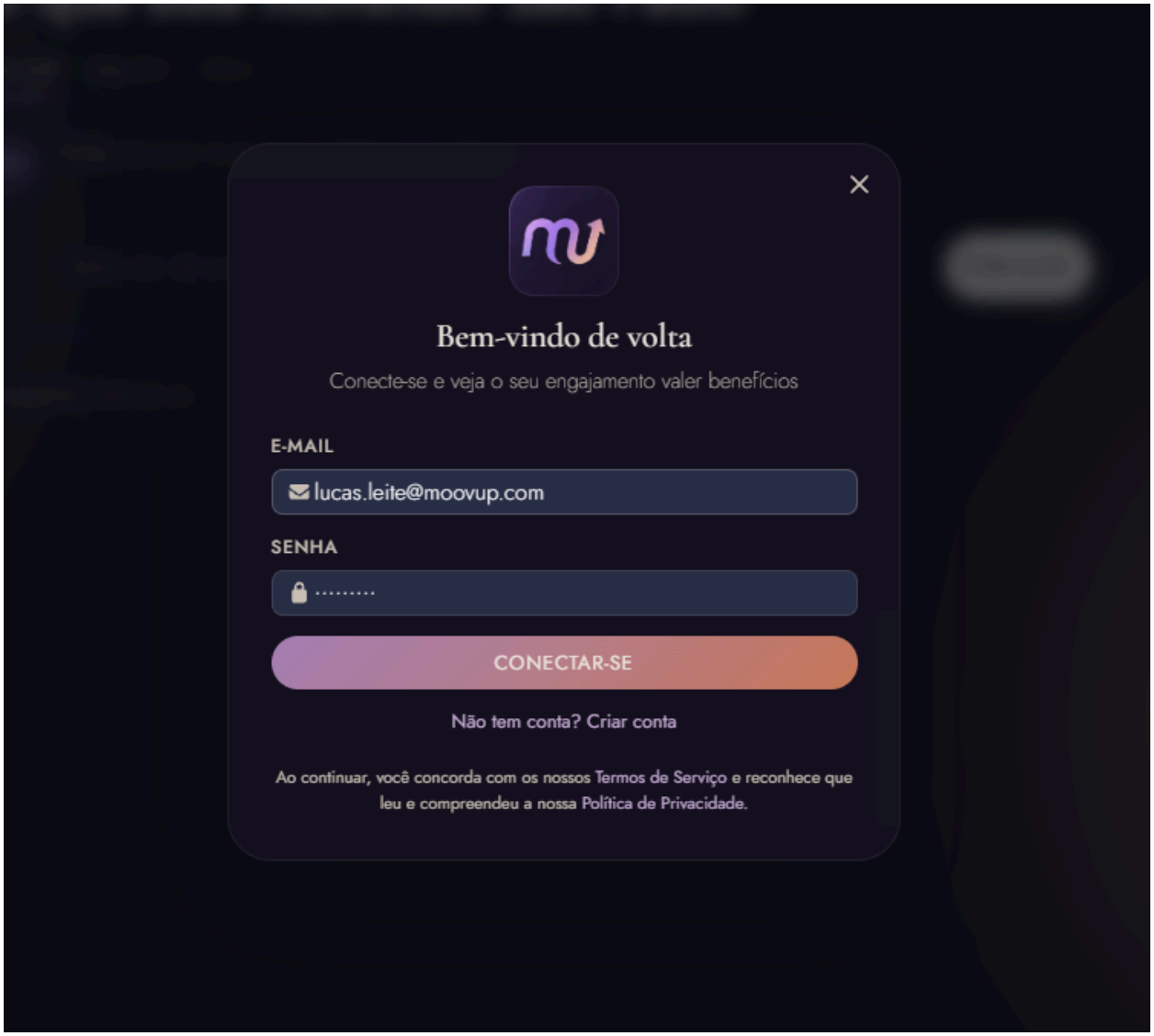


Figura 7: Login. *O usuário entra com e-mail e senha (POST /usuarios/login). Se o e-mail ou a senha estiverem errados, a API devolve 401.*

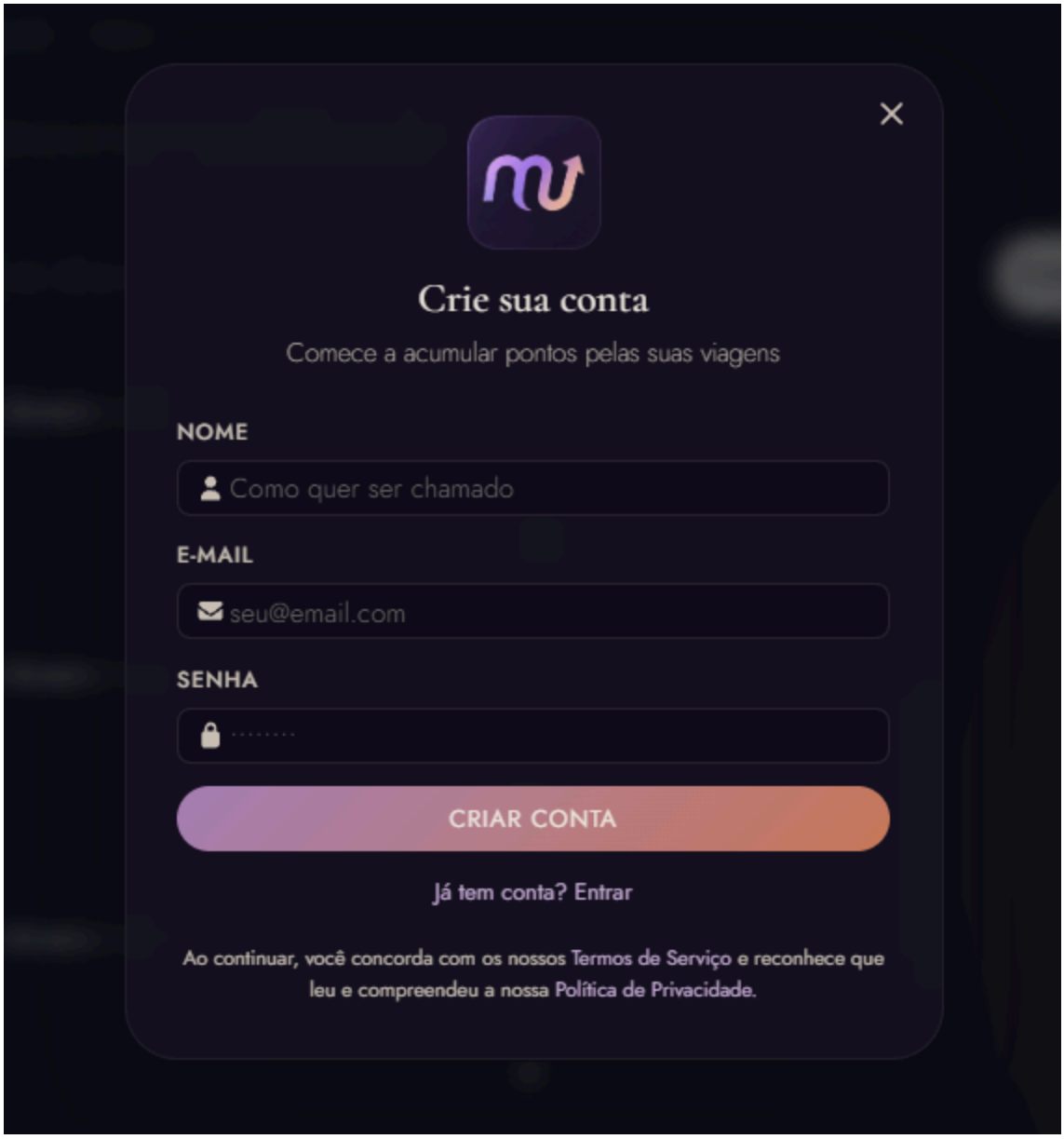


Figura 8: Cadastro. *Cria a conta com nome, e-mail e senha (POST /usuarios). A API valida os campos e recusa e-mail ou nome já cadastrados (422).*

Modelo Entidade-Relacionamento

O modelo tem oito tabelas: USUARIO_MOOVUP, POSTAGEM, COMENTARIO, CURTIDA, SEGUIDOR, TICKET_DIGITAL, VIAGEM e IMPACTO_AMBIENTAL_MOOVPASS. O diagrama foi gerado no Oracle Data Modeler e mostra as colunas de cada tabela com o tipo de dado, as chaves primárias, as chaves estrangeiras e os relacionamentos. Cada tabela corresponde a uma classe da camada models.

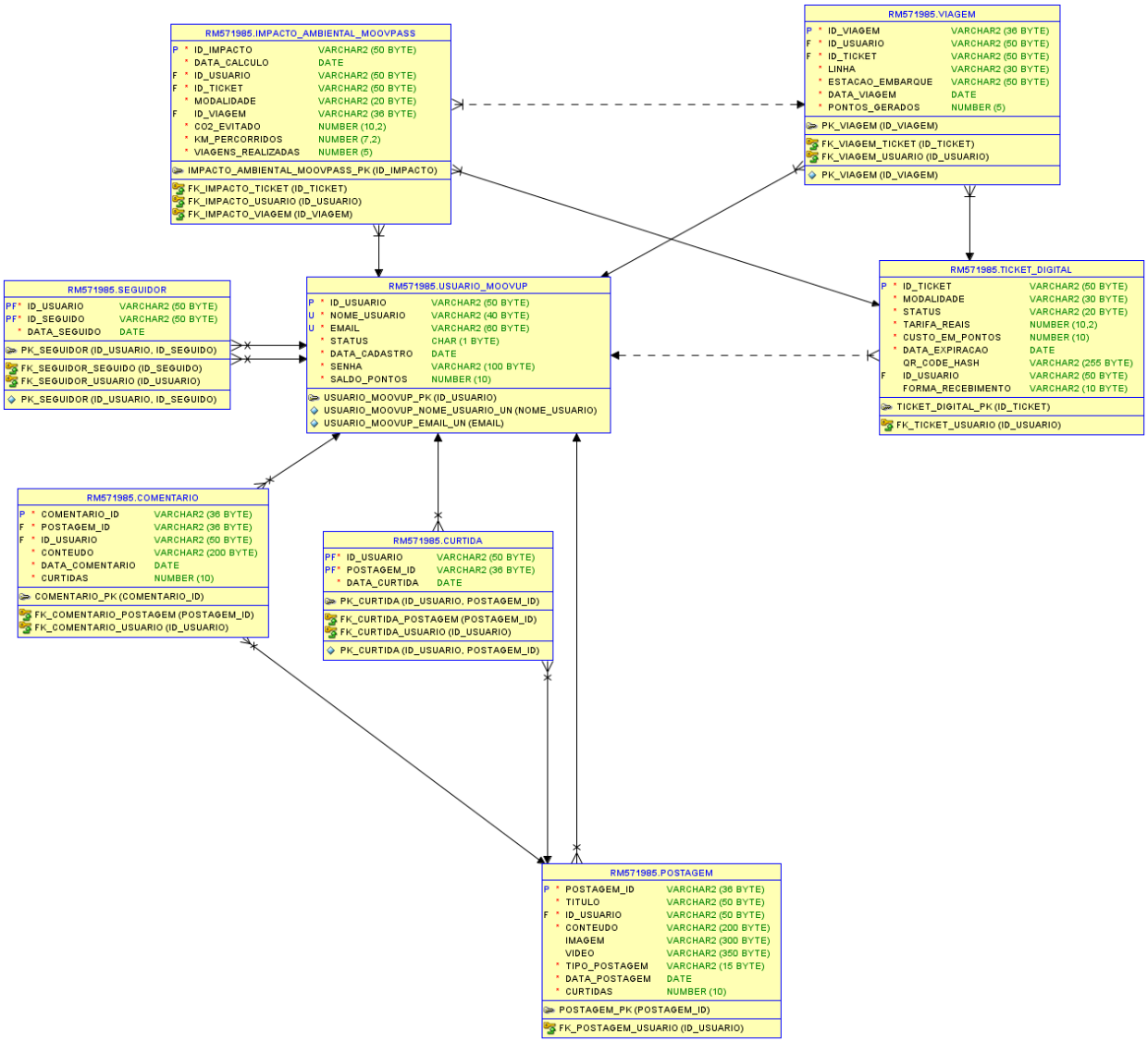
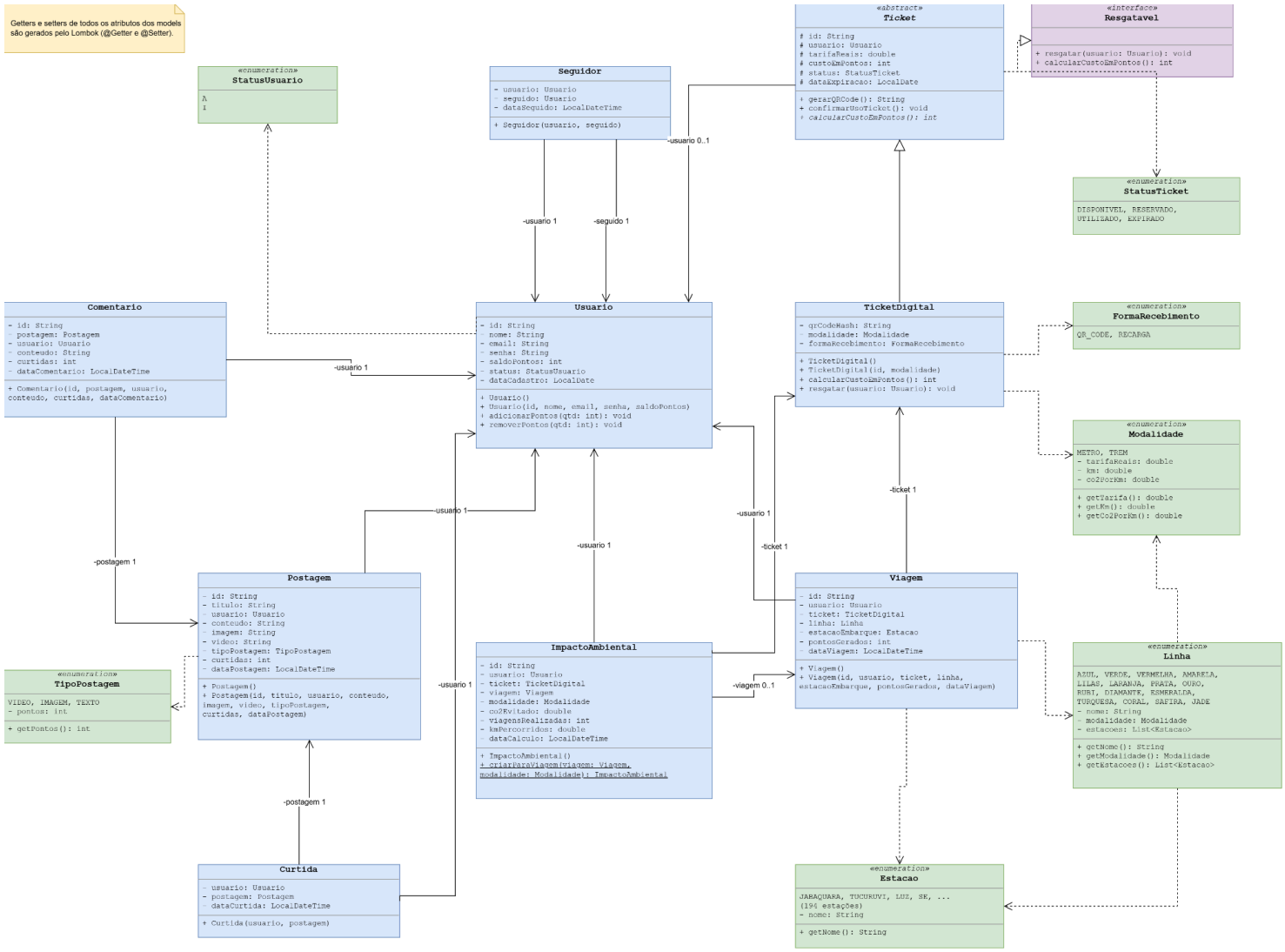
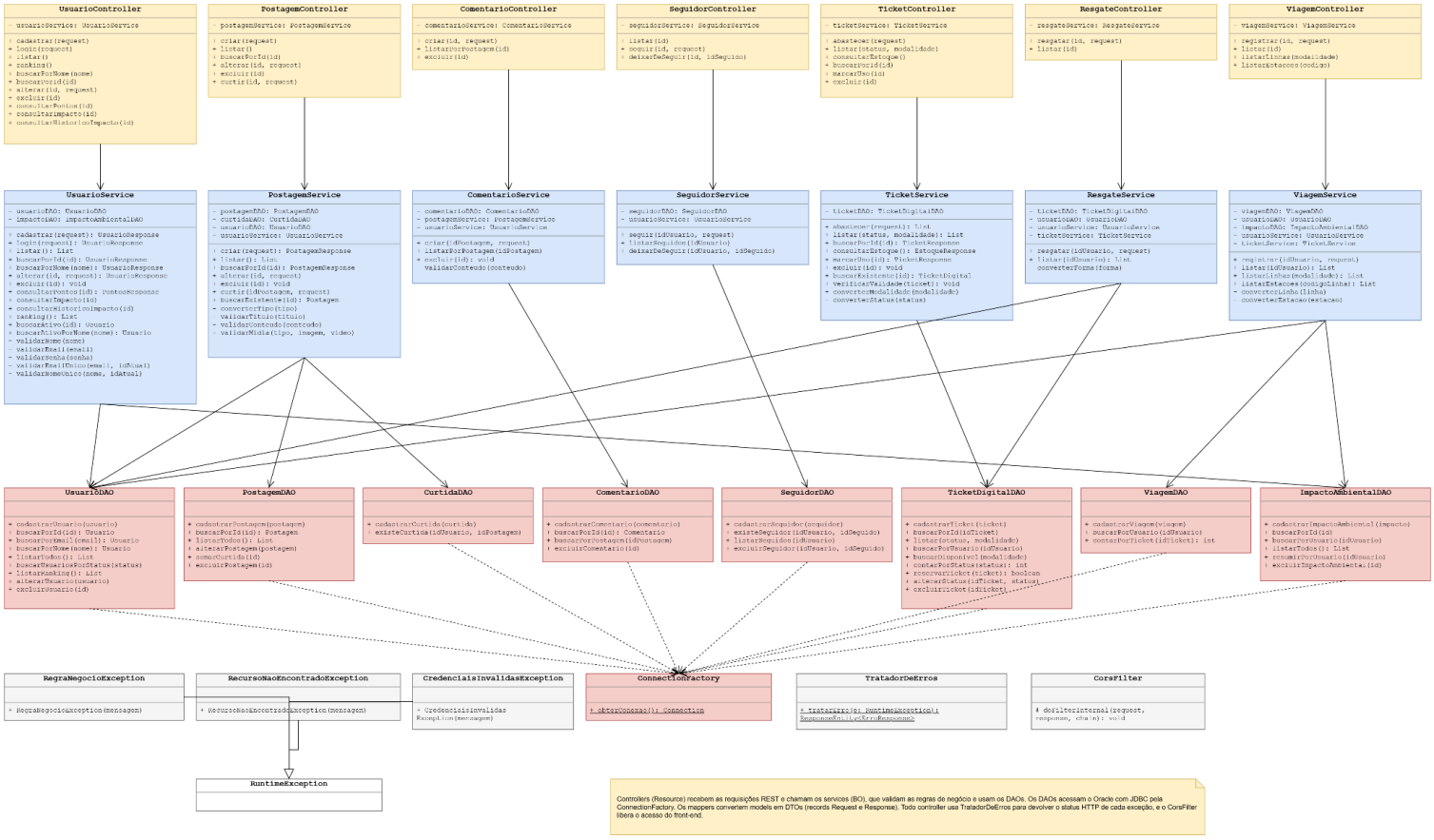

RM571985.IMPACTO_AMBIENTAL_MOOVPASS
P * ID_IMPACTO VARCHAR2 (50 BYTE)
* DATA_CALCULO DATE
F * ID_USUARIO VARCHAR2 (50 BYTE)
F * ID_TICKET VARCHAR2 (50 BYTE)
* MODALIDADE VARCHAR2 (20 BYTE)
F ID_VIAGEM VARCHAR2 (36 BYTE)
* CO2_EVITADO NUMBER (10,2)
* KM_PERCORRIDOS NUMBER (7,2)
* VIAGENS_REALIZADAS NUMBER (5)
IMPACTO_AMBIENTAL_MOOVPASS_PK (ID_IMPACTO)
FK_IMPACTO_TICKET (ID_TICKET)
FK_IMPACTO_USUARIO (ID_USUARIO)
FK_IMPACTO_VIAGEM (ID_VIAGEM)
RM571985.VIAGEM
P * ID_VIAGEM VARCHAR2 (36 BYTE)
F * ID_USUARIO VARCHAR2 (50 BYTE)
F * ID_TICKET VARCHAR2 (50 BYTE)
* LINHA VARCHAR2 (30 BYTE)
* ESTACAO_EMBARQUE VARCHAR2 (50 BYTE)
* DATA_VIAGEM DATE
* PONTOS_GERADOS NUMBER (5)
PK_VIAGEM (ID_VIAGEM)
FK_VIAGEM_TICKET (ID_TICKET)
FK_VIAGEM_USUARIO (ID_USUARIO)
PK_VIAGEM (ID_VIAGEM)
RM571985.SEGUIDOR
PF* ID_USUARIO VARCHAR2 (50 BYTE)
PF* ID_SEGUIDO VARCHAR2 (50 BYTE)
* DATA_SEGUIDO DATE
PK_SEGUIDOR (ID_USUARIO, ID_SEGUIDO)
FK_SEGUIDOR_SEGUIDO (ID_SEGUIDO)
FK_SEGUIDOR_USUARIO (ID_USUARIO)
PK_SEGUIDOR (ID_USUARIO, ID_SEGUIDO)
RM571985.USUARIO_MOOVUP
P * ID_USUARIO VARCHAR2 (50 BYTE)
U * NOME_USUARIO VARCHAR2 (40 BYTE)
U * EMAIL VARCHAR2 (60 BYTE)
* STATUS CHAR (1 BYTE)
* DATA_CADASTRO DATE
* SENHA VARCHAR2 (100 BYTE)
* SALDO_PONTOS NUMBER (10)
USUARIO_MOOVUP_PK (ID_USUARIO)
USUARIO_MOOVUP_NOME_USUARIO_UN (NOME_USUARIO)
USUARIO_MOOVUP_EMAIL_UN (EMAIL)
RM571985.TICKET_DIGITAL
P * ID_TICKET VARCHAR2 (50 BYTE)
* MODALIDADE VARCHAR2 (30 BYTE)
* STATUS VARCHAR2 (20 BYTE)
* TARIFA_REAIS NUMBER (10,2)
* CUSTO_EM_PONTOS NUMBER (10)
* DATA_EXPIRACAO DATE
QR_CODE_HASH VARCHAR2 (255 BYTE)
F ID_USUARIO VARCHAR2 (50 BYTE)
FORMA_RECEBIMENTO VARCHAR2 (10 BYTE)
TICKET_DIGITAL_PK (ID_TICKET)
FK_TICKET_USUARIO (ID_USUARIO)
RM571985.COMENTARIO
P * COMENTARIO_ID VARCHAR2 (36 BYTE)
F * POSTAGEM_ID VARCHAR2 (36 BYTE)
F * ID_USUARIO VARCHAR2 (50 BYTE)
* CONTEUDO VARCHAR2 (200 BYTE)
* DATA_COMENTARIO DATE
* CURTIDAS NUMBER (10)
COMENTARIO_PK (COMENTARIO_ID)
FK_COMENTARIO_POSTAGEM (POSTAGEM_ID)
FK_COMENTARIO_USUARIO (ID_USUARIO)
RM571985.CURTIDA
PF* ID_USUARIO VARCHAR2 (50 BYTE)
PF* POSTAGEM_ID VARCHAR2 (36 BYTE)
* DATA_CURTIDA DATE
PK_CURTIDA (ID_USUARIO, POSTAGEM_ID)
FK_CURTIDA_POSTAGEM (POSTAGEM_ID)
FK_CURTIDA_USUARIO (ID_USUARIO)
PK_CURTIDA (ID_USUARIO, POSTAGEM_ID)
RM571985.POSTAGEM
P * POSTAGEM_ID VARCHAR2 (36 BYTE)
* TITULO VARCHAR2 (50 BYTE)
F * ID_USUARIO VARCHAR2 (50 BYTE)
* CONTEUDO VARCHAR2 (200 BYTE)
IMAGEM VARCHAR2 (300 BYTE)
VIDEO VARCHAR2 (350 BYTE)
* TIPO_POSTAGEM VARCHAR2 (15 BYTE)
* DATA_POSTAGEM DATE
* CURTIDAS NUMBER (10)
POSTAGEM_PK (POSTAGEM_ID)
FK_POSTAGEM_USUARIO (ID_USUARIO)

Diagrama de classes

O diagrama está dividido em duas partes. A primeira mostra a camada de domínio: os models, a classe abstrata Ticket, a interface Resgatavel e os enums, com atributos, métodos, herança, realização e associações. A segunda mostra as camadas da API: os controllers (Resource), os services (BO), os DAOs, a ConnectionFactory, as exceções, o TratadorDeErros e o CorsFilter.





Como executar a aplicação

Ferramentas e versões

- IDE: IntelliJ IDEA
- JDK: 25
- Spring Boot: 4.1.1, com Maven (o projeto inclui o Maven Wrapper, mvnw)
- Banco de dados: Oracle, acessado por JDBC (driver ojdbc17)
- Documentação da API: springdoc-openapi (Swagger UI)

Passo a passo

1. Descompactar o arquivo entregue e abrir a pasta MoovUp no IntelliJ IDEA.
2. Aguardar o IntelliJ baixar as dependências do Maven.
3. Executar a classe MoovUpApplication (pacote br.com.fiap.MoovUp). Outra opção é rodar o comando mvnw spring-boot:run na pasta do projeto.
4. Acessar http://localhost:8080/swagger-ui.html para ver e testar todos os endpoints.

A conexão com o banco Oracle da FIAP já está configurada na classe ConnectionFactory, e o banco já tem dados de exemplo. Para testar o login, use um dos usuários de exemplo, como

lucas.leite@moovup.com com a senha Moovup123.

Deploy

A API está publicada no Render, a partir do repositório no GitHub, com um Dockerfile que compila o projeto com o Maven e executa o arquivo .jar com o Java 25.

- URL: https://challenge-2026-java-sprint-4.onrender.com
- Swagger: https://challenge-2026-java-sprint-4.onrender.com/swagger-ui.html

O serviço está no plano gratuito do Render, que desliga a aplicação depois de 15 minutos sem uso. Por isso, a primeira chamada depois de um período parado pode levar até um minuto e meio.